

```
In [9]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
In [8]: df = pd.read_csv('Walmart.csv')
```

```
In [9]: df.head()
```

```
Out[9]:
```

	Store	Date	Weekly_Sales	Holiday_Flag	Temperature	Fuel_Price	CPI	Unemplo
0	1	05-02-2010	1643690.90	0	42.31	2.572	211.096358	
1	1	12-02-2010	1641957.44	1	38.51	2.548	211.242170	
2	1	19-02-2010	1611968.17	0	39.93	2.514	211.289143	
3	1	26-02-2010	1409727.59	0	46.63	2.561	211.319643	
4	1	05-03-2010	1554806.68	0	46.50	2.625	211.350143	

```
In [4]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6435 entries, 0 to 6434
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Store           6435 non-null  int64
1   Date            6435 non-null  object
2   Weekly_Sales    6435 non-null  float64
3   Holiday_Flag    6435 non-null  int64
4   Temperature     6435 non-null  float64
5   Fuel_Price      6435 non-null  float64
6   CPI             6435 non-null  float64
7   Unemployment    6435 non-null  float64
dtypes: float64(5), int64(2), object(1)
memory usage: 402.3+ KB
```

```
In [105]: df.columns
```

```
Out[105]: Index(['SR NO', 'invoice_ID', 'Branch', 'City', 'category', 'unit_price',  
               'quantity', 'date', 'time', 'payment_method', 'Custom', 'rating',  
               'profit_margin', 'Total_Sale'],  
              dtype='object')
```

```
In [5]: df.duplicated().sum()
```

```
Out[5]: 0
```

```
In [6]: df['Date'] = pd.to_datetime(df['Date'])  
print(df['Date'].dtypes)
```

```
datetime64[ns]
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_86120\1462656412.py:1: UserWarning: Parsing dates in DD/MM/YYYY format when dayfirst=False (the default) was specified. This may lead to inconsistently parsed dates! Specify a format to ensure consistent parsing.  
df['Date'] = pd.to_datetime(df['Date'])
```

```
In [ ]: import warnings  
  
warnings.filterwarnings('ignore')  
  
from sqlalchemy import create_engine
```

```
In [ ]: import pandas as pd  
  
engine = create_engine("mysql+mysqldb://root:Balaji%40123@127.0.0.1:3306/walmart")
```

```
In [ ]: conn = engine.connect()
```

```
In [ ]: df.to_sql('walmart', engine, index=False, if_exists='replace')
```

```
In [ ]: //////////////////////////////////////
```

```
In [3]: import pandas as pd  
from sqlalchemy import create_engine
```

```
In [5]: path = r"C:\Users\Admin\OneDrive\Desktop\Walmart_sale_python.xlsx"
```

```
In [4]: import pandas as pd
```

```
In [7]: df = pd.read_excel(path)
```

```
In [8]: df.head()
```

```
Out[8]:
```

	SR NO	invoice_ID	Branch	City	category	unit_price	quantity	date	payme
0	1	WALM003-SAN ANTONIO	WALM003	SAN ANTONIO	Health and beauty	74.69	7	2019-01-05	
1	2	WALM048-HARLINGEN	WALM048	HARLINGEN	Electronic accessories	15.28	5	2019-03-08	
2	3	WALM067-HALTOM CITY	WALM067	HALTOM CITY	Home and lifestyle	46.33	7	2019-03-03	
3	4	WALM064-BEDFORD	WALM064	BEDFORD	Health and beauty	58.22	8	2019-01-27	
4	5	WALM013-IRVING	WALM013	IRVING	Sports and travel	86.31	7	2019-02-08	



```
In [9]: df.isnull().sum()
```

```
Out[9]: SR NO      0
invoice_ID      0
Branch          0
City            0
category        0
unit_price      0
quantity        0
date            0
payment_method  0
payment mode    0
rating          0
profit_margin   0
Total_Sale      0
dtype: int64
```

```
In [10]: df.columns
```

```
Out[10]: Index(['SR NO', 'invoice_ID', 'Branch', 'City', 'category', 'unit_price',
               'quantity', 'date', 'payment_method', 'payment mode', 'rating',
               'profit_margin', 'Total_Sale'],
              dtype='object')
```

```
In [13]: df.drop_duplicates(inplace=True)
```

```
In [14]: df.head()
```

```
Out[14]:
```

	SR NO	invoice_ID	Branch	City	category	unit_price	quantity	date	payme
0	1	WALM003-SAN ANTONIO	WALM003	SAN ANTONIO	Health and beauty	74.69	7	2019- 01-05	
1	2	WALM048- HARLINGEN	WALM048	HARLINGEN	Electronic accessories	15.28	5	2019- 03-08	
2	3	WALM067- HALTOM CITY	WALM067	HALTOM CITY	Home and lifestyle	46.33	7	2019- 03-03	
3	4	WALM064- BEDFORD	WALM064	BEDFORD	Health and beauty	58.22	8	2019- 01-27	
4	5	WALM013- IRVING	WALM013	IRVING	Sports and travel	86.31	7	2019- 02-08	

```
In [16]: df['date'] = pd.to_datetime(df['date'])
```

```
In [17]: df['day'] = df['date'].dt.day  
df['month'] = df['date'].dt.month  
df['year'] = df['date'].dt.year
```

```
In [18]: df.rename(columns={'invoice_ID': 'InvoiceID'}, inplace=True)
```

```
In [19]: df['payment_method'] = df['payment_method'].replace({'Ewallet': 'E-Wallet'}
```

```
In [20]: df[df['rating'] > 8]
```

Out[20]:

	SR NO	InvoiceID	Branch	City	category	unit_price	quantity	date	p
0	1	WALM003-SAN ANTONIO	WALM003	SAN ANTONIO	Health and beauty	74.69	7	2019-01-05	
1	2	WALM048-HARLINGEN	WALM048	HARLINGEN	Electronic accessories	15.28	5	2019-03-08	
3	4	WALM064-BEDFORD	WALM064	BEDFORD	Health and beauty	58.22	8	2019-01-27	
13	14	WALM035-SAN ANGELO	WALM035	SAN ANGELO	Food and beverages	43.19	10	2019-02-07	
18	19	WALM053-CONROE	WALM053	CONROE	Food and beverages	54.67	3	2019-01-21	
...	...	...	...	...	...	...	...	...	...
5208	5209	WALM042-BRYAN	WALM042	BRYAN	Home and lifestyle	28.00	3	2023-08-28	
5210	5211	WALM048-HARLINGEN	WALM048	HARLINGEN	Home and lifestyle	37.00	2	2021-08-21	
5219	5220	WALM066-GRAPEVINE	WALM066	GRAPEVINE	Fashion accessories	25.00	1	2023-10-07	
5225	5226	WALM042-BRYAN	WALM042	BRYAN	Fashion accessories	51.00	3	2022-11-16	
5233	5234	WALM015-GRAND PRAIRIE	WALM015	GRAND PRAIRIE	Fashion accessories	80.00	3	2022-12-20	

1003 rows × 16 columns



```
In [21]: df.describe()
```

Out[21]:

	SR NO	unit_price	quantity	rating	profit_margin	Total_Sale
count	10000.00000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	5000.50000	50.539613	2.351400	5.830170	0.393813	121.047038
std	2886.89568	21.222542	1.604735	1.764097	0.090656	112.633335
min	1.00000	10.080000	1.000000	3.000000	0.180000	10.170000
25%	2500.75000	32.000000	1.000000	4.000000	0.330000	54.000000
50%	5000.50000	51.000000	2.000000	6.000000	0.330000	87.300000
75%	7500.25000	68.710000	3.000000	7.000000	0.480000	156.000000
max	10000.00000	99.960000	10.000000	10.000000	0.570000	993.000000



```
In [22]: # 12. Group by category  
df.groupby('category')['Total_Sale'].sum()
```

```
Out[22]: category  
Electronic accessories      78175.03  
Fashion accessories         489840.90  
Food and beverages          53471.28  
Health and beauty           46851.18  
Home and lifestyle          489634.06  
Sports and travel           52497.93  
Name: Total_Sale, dtype: float64
```

```
In [23]: # 13. Average unit price per branch  
df.groupby('Branch')['unit_price'].mean()
```

```
Out[23]: Branch  
WALM001      52.917200  
WALM002      51.376000  
WALM003      51.731774  
WALM004      53.758333  
WALM005      50.590476  
...  
WALM096      56.228875  
WALM097      45.141194  
WALM098      56.200152  
WALM099      48.518303  
WALM100      54.419355  
Name: unit_price, Length: 100, dtype: float64
```

```
In [24]: # 14. Count of transactions per city  
df['City'].value_counts()
```

```
Out[24]: WESLACO      396  
WAXAHACHIE      378  
PORT ARTHUR      239  
PLANO            237  
RICHARDSON       230  
...  
IRVING           57  
LEWISVILLE      56  
COLLEGE STATION  56  
AMARILLO         52  
LAKE JACKSON     51  
Name: City, Length: 98, dtype: int64
```

```
In [25]: ▶ # 15. Max sale per category
df.groupby('category')['Total_Sale'].max()
```

```
Out[25]: category
Electronic accessories    897.57
Fashion accessories       993.00
Food and beverages        985.20
Health and beauty         905.00
Home and lifestyle        975.00
Sports and travel         954.40
Name: Total_Sale, dtype: float64
```

```
In [26]: ▶ # 16. Min rating per payment method
df.groupby('payment_method')['rating'].min()
```

```
Out[26]: payment_method
Cash          3.0
Credit card   3.0
E-Wallet       3.0
Name: rating, dtype: float64
```

```
In [27]: ▶ # 17. Total quantity sold per month
df.groupby('month')['quantity'].sum()
```

```
Out[27]: month
1      2515
2      2242
3      2440
4       544
5       606
6       513
7       579
8      1379
9      1326
10     1453
11     4854
12     5063
Name: quantity, dtype: int64
```

```
In [28]: ▶ # 18. Profit margin by category
df.groupby('category')['profit_margin'].mean()
```

```
Out[28]: category
Electronic accessories    0.393150
Fashion accessories       0.392965
Food and beverages        0.400345
Health and beauty         0.400263
Home and lifestyle        0.394683
Sports and travel         0.382229
Name: profit_margin, dtype: float64
```

```
In [29]: # 19. Correlation matrix
df.corr()
```

C:\Users\Admin\AppData\Local\Temp\ipykernel_7640\3060496249.py:2: FutureWarning: The default value of numeric_only in DataFrame.corr is deprecated. In a future version, it will default to False. Select only valid columns or specify the value of numeric_only to silence this warning.

```
df.corr()
```

Out[29]:

	SR NO	unit_price	quantity	rating	profit_margin	Total_Sale	day
SR NO	1.000000	-0.038075	-0.337925	-0.402733	-0.112003	-0.285009	0.003957
unit_price	-0.038075	1.000000	0.064845	0.018382	-0.011998	0.532051	-0.007888
quantity	-0.337925	0.064845	1.000000	0.135715	0.001472	0.800607	-0.019169
rating	-0.402733	0.018382	0.135715	1.000000	0.065824	0.110156	0.000849
profit_margin	-0.112003	-0.011998	0.001472	0.065824	1.000000	-0.002448	-0.011337
Total_Sale	-0.285009	0.532051	0.800607	0.110156	-0.002448	1.000000	-0.016173
day	0.003957	-0.007888	-0.019169	0.000849	-0.011337	-0.016173	1.000000
month	0.486909	-0.040224	-0.386160	-0.237748	-0.033882	-0.323396	0.005166
year	0.297220	-0.037160	-0.374436	-0.126135	0.011822	-0.310918	0.010860

```
In [30]: # 20. Pivot table: category vs city
pd.pivot_table(df, values='Total_Sale', index='City', columns='category',
```

Out[30]:

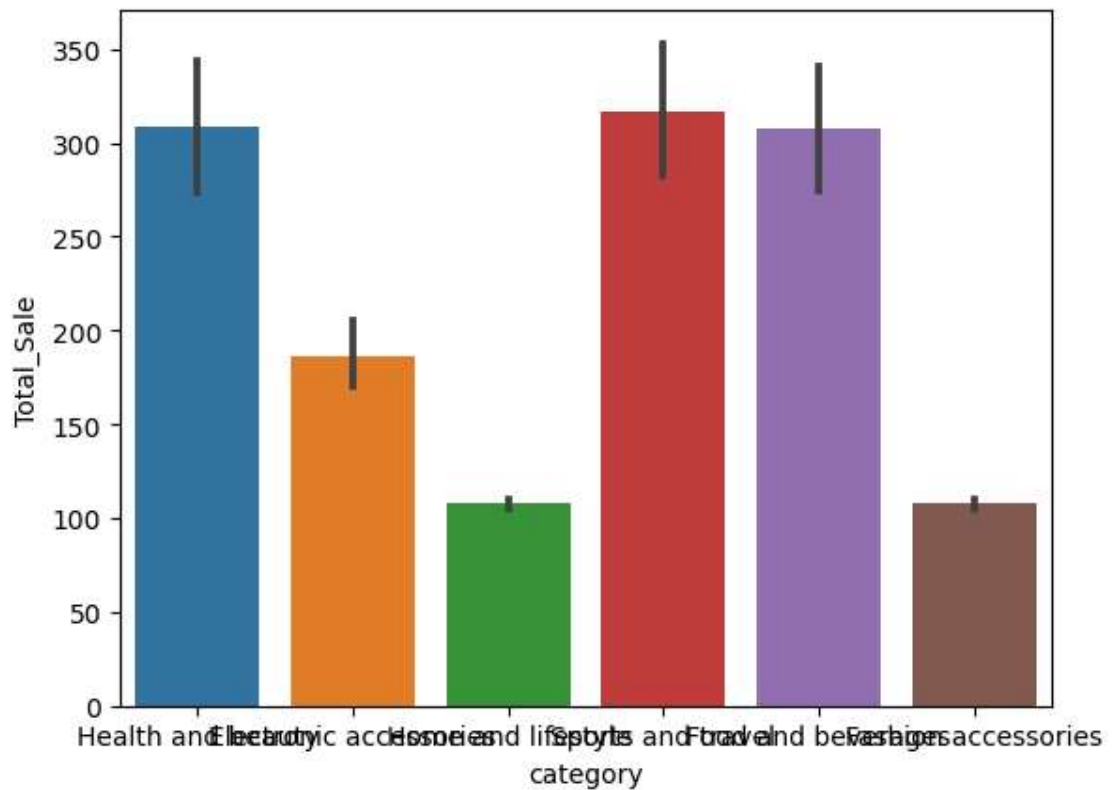
category	Electronic accessories	Fashion accessories	Food and beverages	Health and beauty	Home and lifestyle	Sports and travel
City						
ABILENE	465.99	4806.21	601.18	42.24	3462.20	NaN
ALAMO	NaN	2470.09	339.57	426.05	3505.86	789.78
ALICE	657.16	2468.00	617.48	NaN	3168.02	466.25
ALLEN	115.36	3674.00	249.96	813.00	2356.60	479.58
AMARILLO	1200.35	2636.48	NaN	878.70	2061.93	NaN
...	...	...	...	...	...	...
VICTORIA	1339.29	6320.78	NaN	NaN	8348.96	NaN
WACO	1597.32	6639.40	887.94	699.64	7046.67	769.20
WAXAHACHIE	4166.90	16104.86	115.80	1593.07	18178.76	543.94
WEATHERFORD	732.26	7840.30	NaN	NaN	8757.04	745.00
WESLACO	3895.05	18618.39	1268.73	120.82	21473.89	974.91

98 rows × 6 columns


```
In [32]: ▶ import matplotlib.pyplot as plt
import seaborn as sns
```

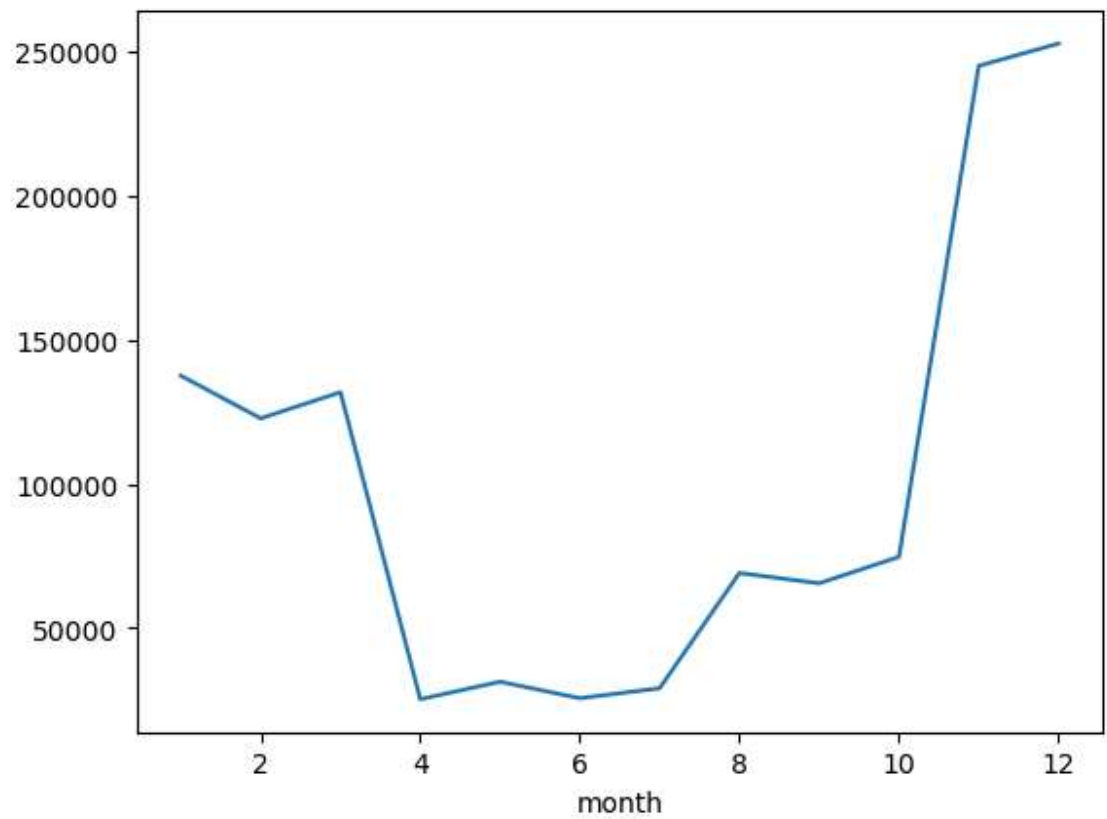
```
In [33]: ▶ # 21. Bar plot of sales by category
sns.barplot(x='category', y='Total_Sale', data=df)
```

```
Out[33]: <Axes: xlabel='category', ylabel='Total_Sale'>
```



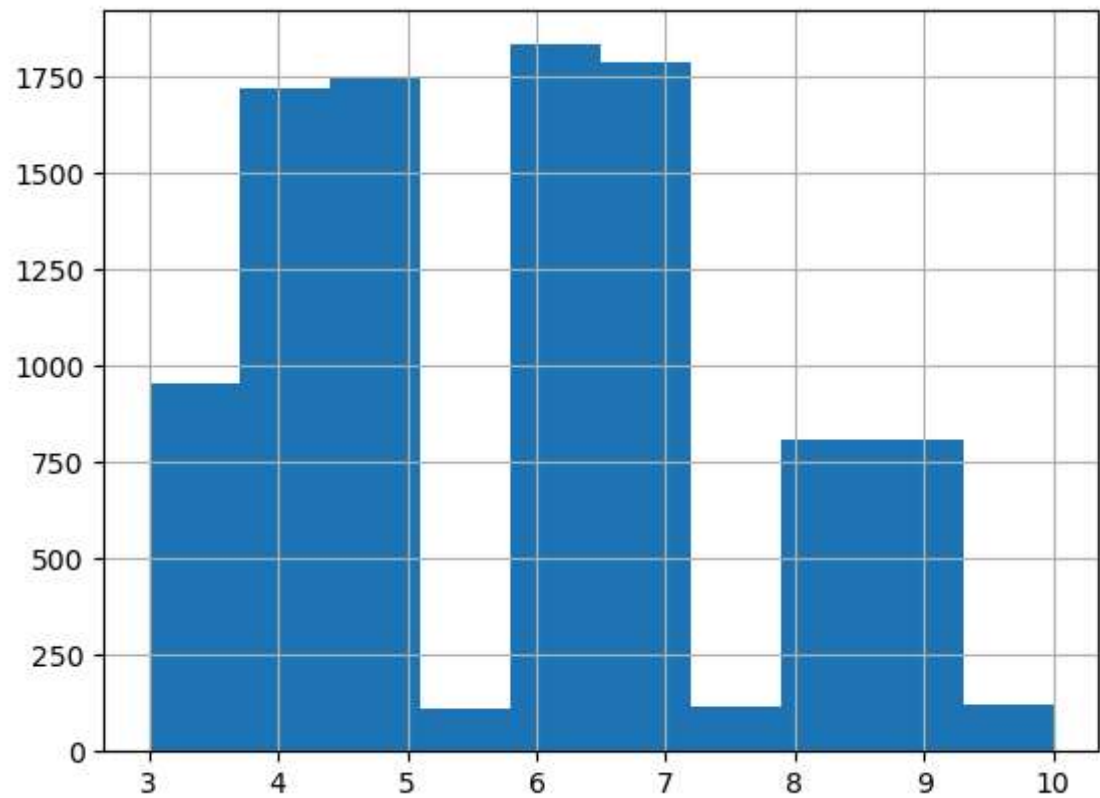
```
In [35]: # 22. Line plot of monthly sales
monthly_sales = df.groupby('month')['Total_Sale'].sum()
monthly_sales.plot(kind='line')
```

Out[35]: <Axes: xlabel='month'>



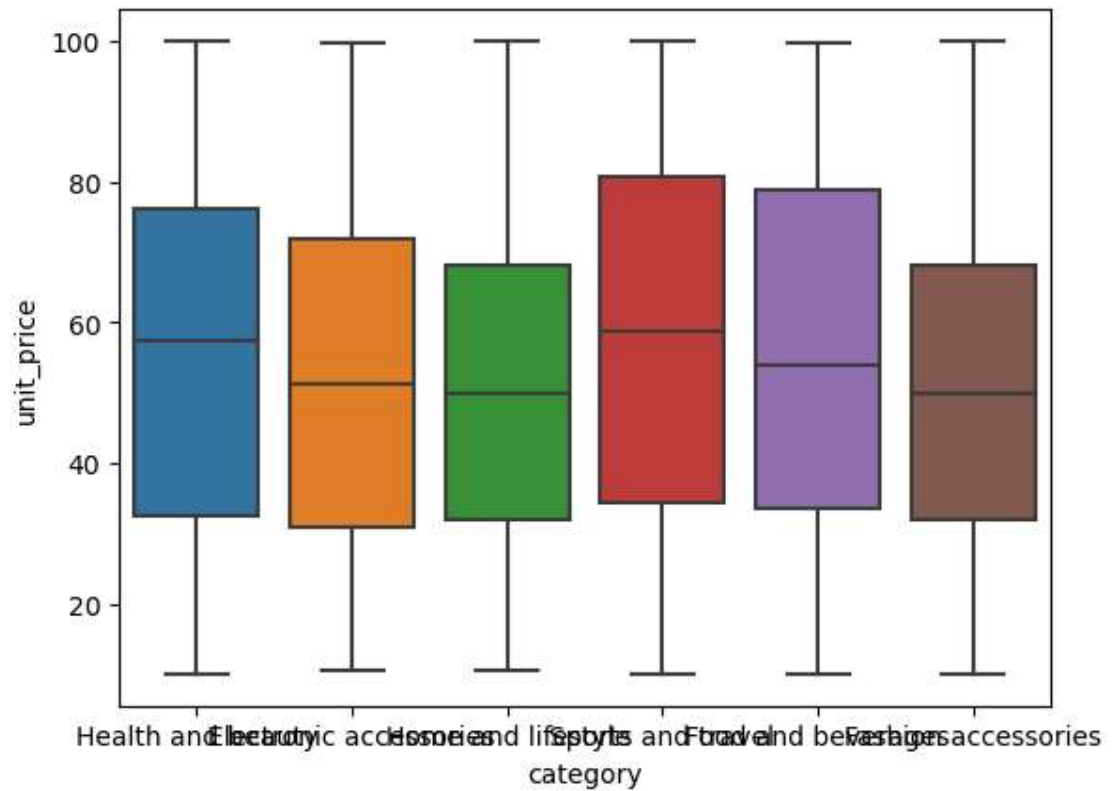
```
In [36]: # 23. Histogram of ratings  
df['rating'].hist()
```

Out[36]: <Axes: >



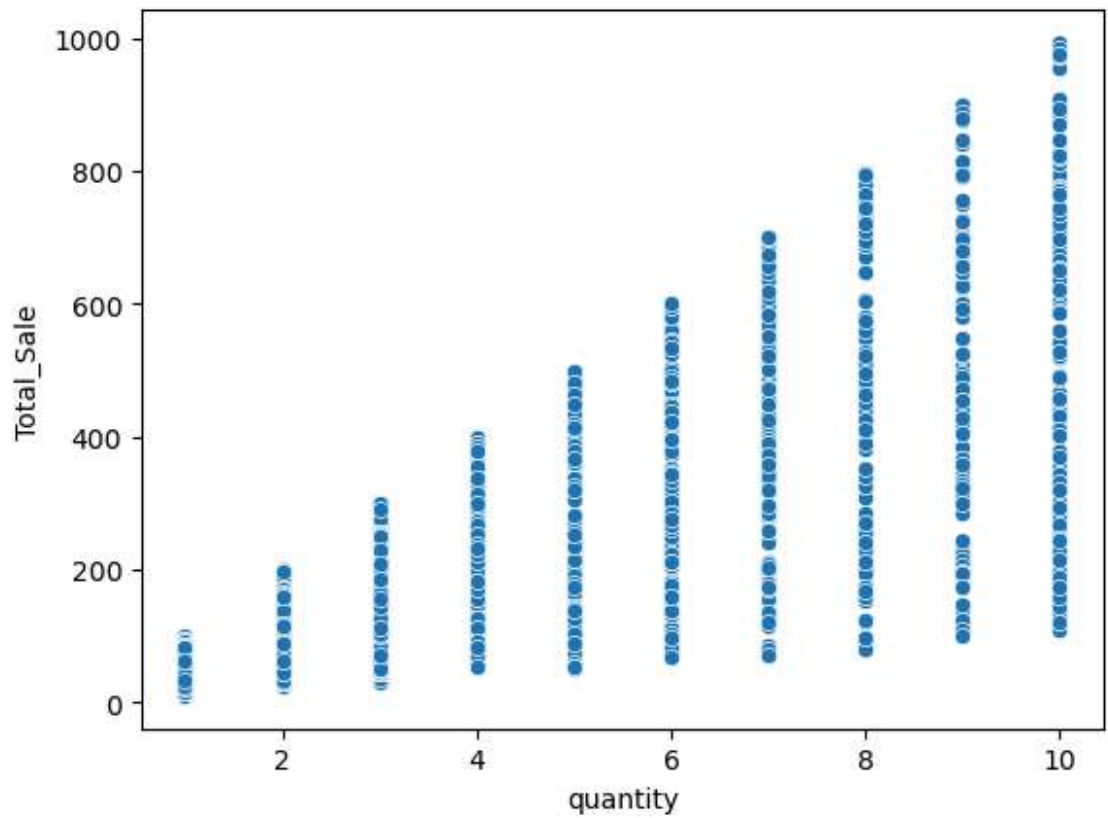
```
In [37]: # 24. Boxplot of unit price by category
sns.boxplot(x='category', y='unit_price', data=df)
```

```
Out[37]: <Axes: xlabel='category', ylabel='unit_price'>
```



```
In [38]: # 25. Scatter plot of quantity vs total sale
sns.scatterplot(x='quantity', y='Total_Sale', data=df)
```

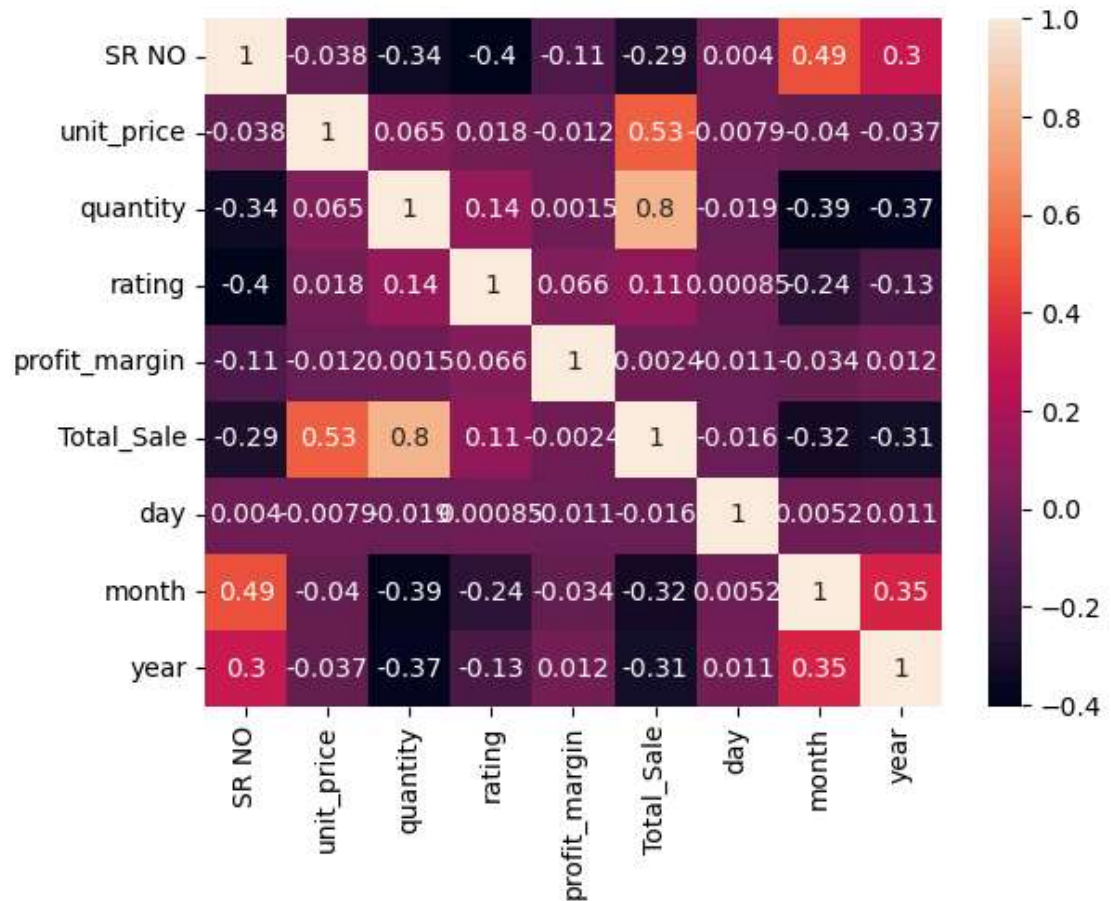
```
Out[38]: <Axes: xlabel='quantity', ylabel='Total_Sale'>
```



```
In [39]: # 26. Heatmap of correlation
sns.heatmap(df.corr(), annot=True)
```

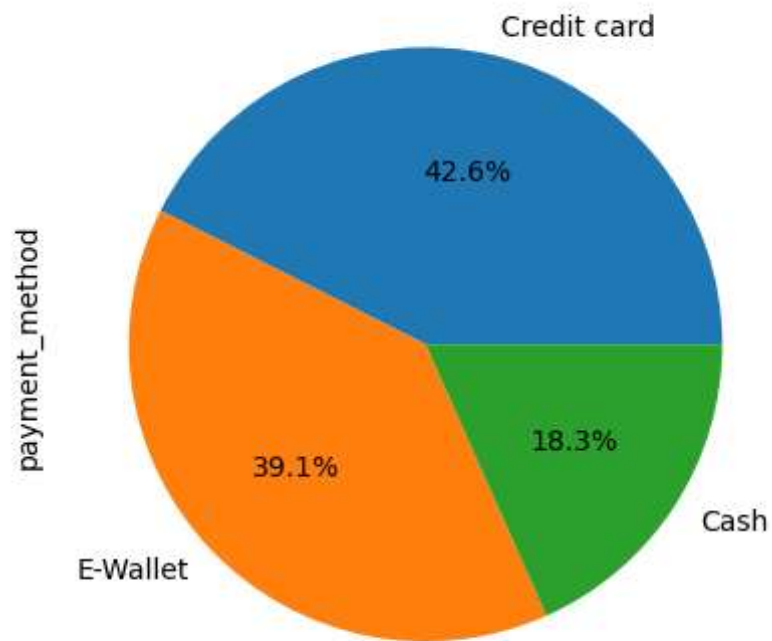
C:\Users\Admin\AppData\Local\Temp\ipykernel_7640\116179910.py:2: FutureWarning: The default value of numeric_only in DataFrame.corr is deprecated. In a future version, it will default to False. Select only valid columns or specify the value of numeric_only to silence this warning.
sns.heatmap(df.corr(), annot=True)

Out[39]: <Axes: >



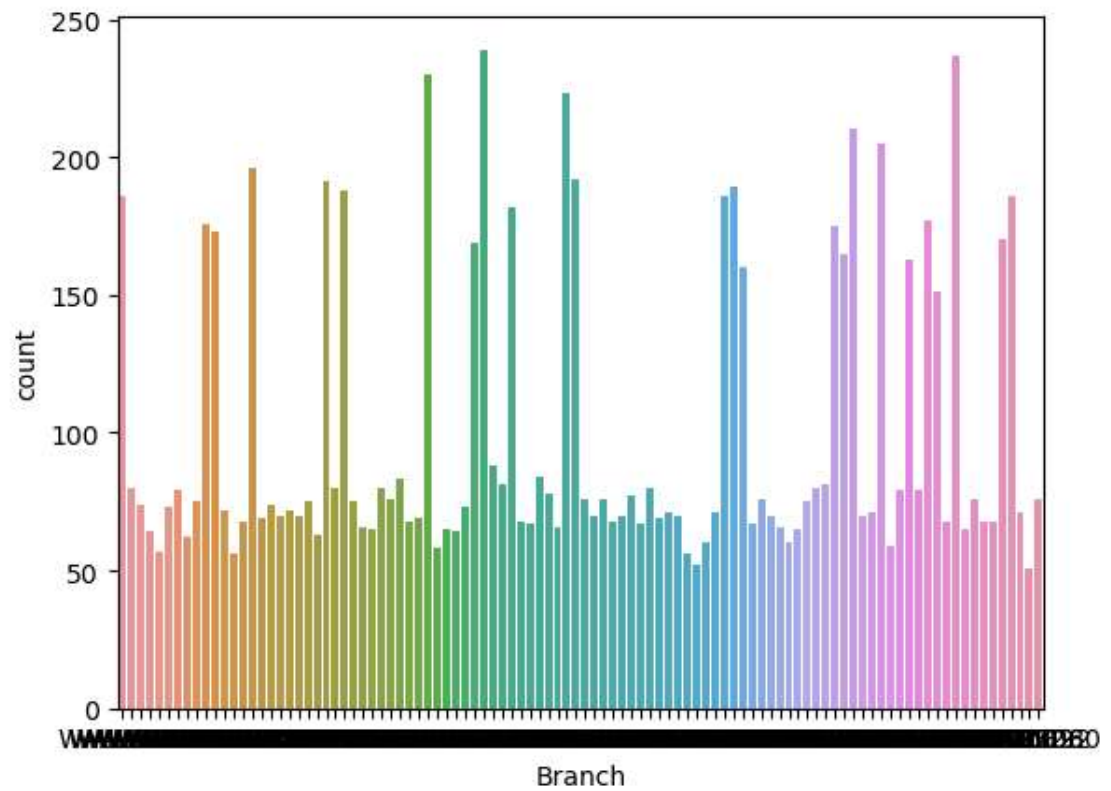
```
In [41]: ▶ # 27. Pie chart of payment methods
df['payment_method'].value_counts().plot.pie(autopct='%1.1f%%')
```

```
Out[41]: <Axes: ylabel='payment_method'>
```



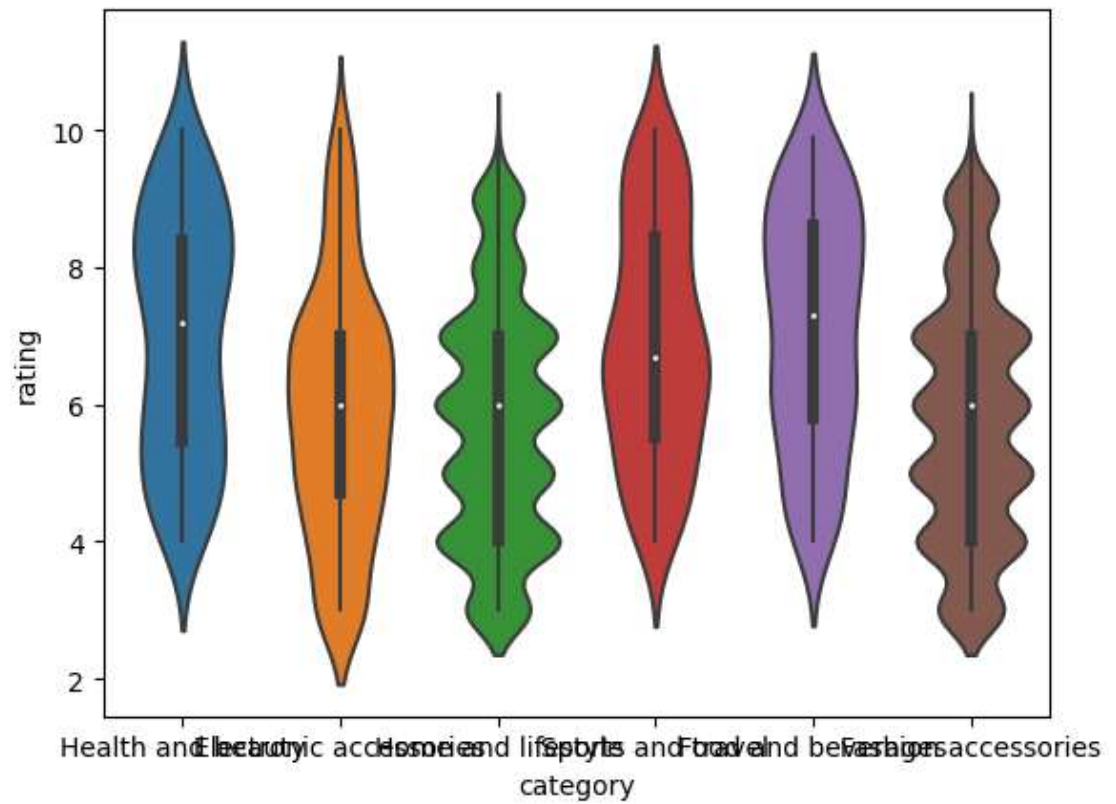
```
In [43]: # 28. Count plot of branches
sns.countplot(x='Branch', data=df)
```

```
Out[43]: <Axes: xlabel='Branch', ylabel='count'>
```



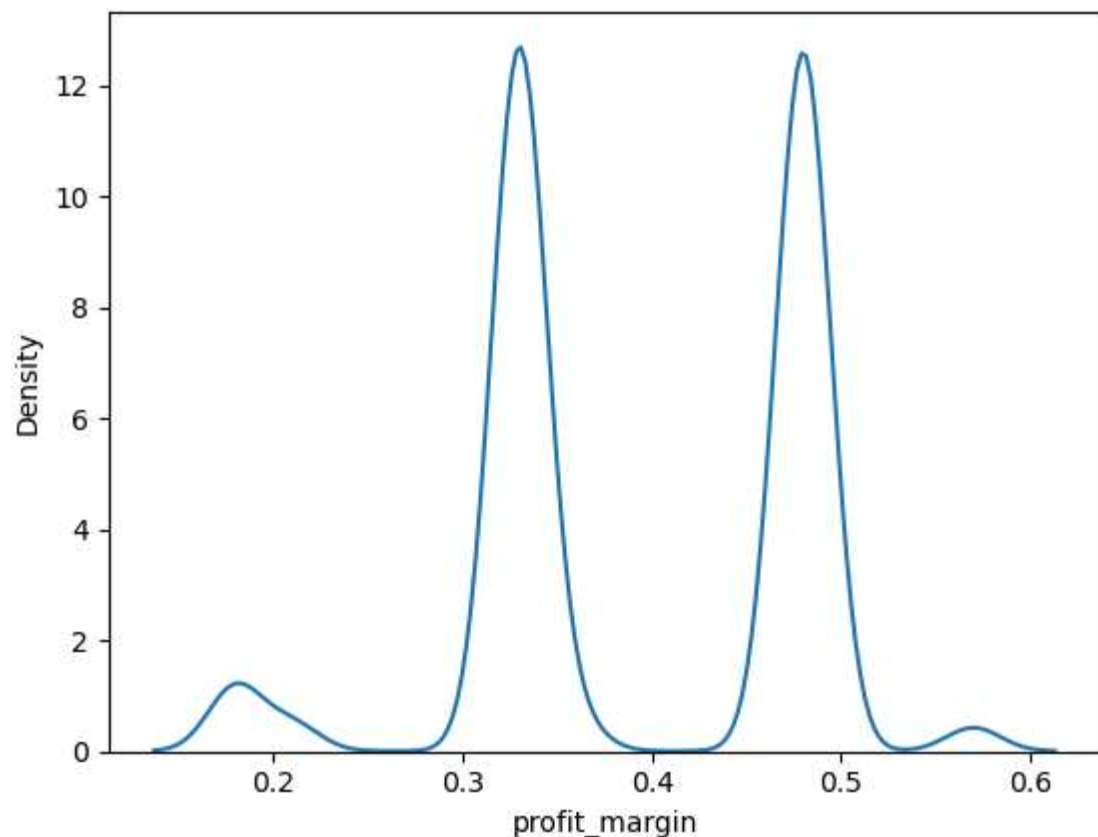

```
In [44]: # 29. Violin plot of rating by category
sns.violinplot(x='category', y='rating', data=df)
```

```
Out[44]: <Axes: xlabel='category', ylabel='rating'>
```



```
In [45]: ▶ # 30. KDE plot of profit margin
sns.kdeplot(df['profit_margin'])
```

```
Out[45]: <Axes: xlabel='profit_margin', ylabel='Density'>
```



```
In [46]: ▶ # 31. Create total profit column
df['Total_Profit'] = df['unit_price'] * df['quantity'] * df['profit_margin']
```

```
In [47]: ▶ # 32. Flag high-value sales
df['High_Sale'] = df['Total_Sale'] > 100
```

```
In [48]: ▶ # 33. Average sale per invoice
df['Avg_Sale'] = df['Total_Sale'] / df['quantity']
```

```
In [49]: ▶ # 34. Weekend vs weekday flag
df['is_weekend'] = df['date'].dt.dayofweek >= 5
```

```
In [51]: ▶ # 36. Most profitable category
df.groupby('category')['Total_Profit'].sum().idxmax()
```

```
Out[51]: 'Fashion accessories'
```

```
In [52]: ▶ # 37. Least profitable city
df.groupby('City')['Total_Profit'].sum().idxmin()
```

```
Out[52]: 'ALICE'
```

```
In [54]: ▶ # 38. Monthly growth rate
monthly = df.groupby('month')['Total_Sale'].sum()
monthly.pct_change()
```

```
Out[54]: month
1         NaN
2    -0.107554
3     0.073735
4    -0.807687
5     0.239661
6    -0.182090
7     0.134628
8     1.369855
9    -0.051284
10    0.139466
11    2.279591
12    0.031470
Name: Total_Sale, dtype: float64
```

```
In [55]: ▶ # 39. Category contribution to total sales
df.groupby('category')['Total_Sale'].sum() / df['Total_Sale'].sum()
```

```
Out[55]: category
Electronic accessories    0.064582
Fashion accessories       0.404670
Food and beverages        0.044174
Health and beauty         0.038705
Home and lifestyle        0.404499
Sports and travel         0.043370
Name: Total_Sale, dtype: float64
```

```
In [56]: ▶ # 40. Rating segmentation
df['rating_segment'] = pd.cut(df['rating'], bins=[0, 5, 8, 10], labels=['L', 'M', 'H'])
```

```
In [57]: ▶ from sqlalchemy import create_engine
```

```
In [58]: ▶ # 41. Connect to MySQL
engine = create_engine("mysql+mysqldb://root:password@localhost:3306/walmart")
```

```
In [59]: ▶ # 43. Export to CSV
df.to_csv('walmart_cleaned.csv', index=False)
```

```
In [96]: ► pip install sqlalchemy pymysql
```

Requirement already satisfied: sqlalchemy in c:\users\admin\anaconda3\lib\site-packages (1.4.39)Note: you may need to restart the kernel to use updated packages.

Requirement already satisfied: pymysql in c:\users\admin\anaconda3\lib\site-packages (1.1.1)

Requirement already satisfied: greenlet!=0.4.17 in c:\users\admin\anaconda3\lib\site-packages (from sqlalchemy) (2.0.1)

```
In [61]: ► import pandas as pd
from sqlalchemy import create_engine
```

```
In [63]: ► path = r"C:\Users\Admin\OneDrive\Desktop\Walmart_sale_python.xlsx"
df = pd.read_excel(path)
```

```
In [66]: ► import pandas as pd
engine = create_engine("mysql+mysqldb://root:Balaji%40123@127.0.0.1:3306/w
conn = engine.connect()
```

```
In [67]: ► df.to_sql(name='walmart_sales', con=engine, index=False, if_exists='replac
```

```
Out[67]: 10000
```

```
In [68]: ► query = "select * from walmart_sales"
```

```
In [69]: ► df=pd.read_sql(query,engine)
```

In [70]: `df`

Out[70]:

	SR NO	invoice_ID	Branch	City	category	unit_price	quantity	date
0	1	WALM003-SAN ANTONIO	WALM003	SAN ANTONIO	Health and beauty	74.69	7	201-01-01
1	2	WALM048-HARLINGEN	WALM048	HARLINGEN	Electronic accessories	15.28	5	201-03-01
2	3	WALM067-HALTOM CITY	WALM067	HALTOM CITY	Home and lifestyle	46.33	7	201-03-01
3	4	WALM064-BEDFORD	WALM064	BEDFORD	Health and beauty	58.22	8	201-01-02
4	5	WALM013-IRVING	WALM013	IRVING	Sports and travel	86.31	7	201-02-01
...	...	...	...	...	...	...	...	...
9995	9996	WALM056-ROWLETT	WALM056	ROWLETT	Fashion accessories	37.00	3	202-08-01
9996	9997	WALM030-RICHARDSON	WALM030	RICHARDSON	Home and lifestyle	58.00	2	202-02-02
9997	9998	WALM050-VICTORIA	WALM050	VICTORIA	Fashion accessories	52.00	3	202-06-01
9998	9999	WALM032-TYLER	WALM032	TYLER	Home and lifestyle	79.00	2	202-02-02
9999	10000	WALM069-ROCKWALL	WALM069	ROCKWALL	Fashion accessories	62.00	3	202-09-02

10000 rows × 13 columns



In [71]: `print(df.columns)`

```
Index(['SR NO', 'invoice_ID', 'Branch', 'City', 'category', 'unit_price',  
      'quantity', 'date', 'payment_method', 'payment mode', 'rating',  
      'profit_margin', 'Total_Sale'],  
      dtype='object')
```

In [72]: `df.head()`

Out[72]:

	SR NO	invoice_ID	Branch	City	category	unit_price	quantity	date	payme
0	1	WALM003-SAN ANTONIO	WALM003	SAN ANTONIO	Health and beauty	74.69	7	2019- 01-05	
1	2	WALM048-HARLINGEN	WALM048	HARLINGEN	Electronic accessories	15.28	5	2019- 03-08	
2	3	WALM067-HALTOM CITY	WALM067	HALTOM CITY	Home and lifestyle	46.33	7	2019- 03-03	
3	4	WALM064-BEDFORD	WALM064	BEDFORD	Health and beauty	58.22	8	2019- 01-27	
4	5	WALM013-IRVING	WALM013	IRVING	Sports and travel	86.31	7	2019- 02-08	



In []: